RAG Assignment: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Roll No.: 2025201029

GitHub: https://github.com/MA9983/rag-biencoder-vs-crossencoder

Document	RFC 6749: The OAuth 2.0 Authorization Framework (IETF), 76 pages
Chunking	Section-wise (Sections 1–10), 132 chunks (~115 words each)
Bi-encoder + store	all-MiniLM-L6-v2 + FAISS (cosine similarity)
Cross-encoder	ms-marco-MiniLM-L-6-v2
LLM	Gemini 3.1 Flash-Lite, same prompt for both pipelines

1. Architecture Comparison

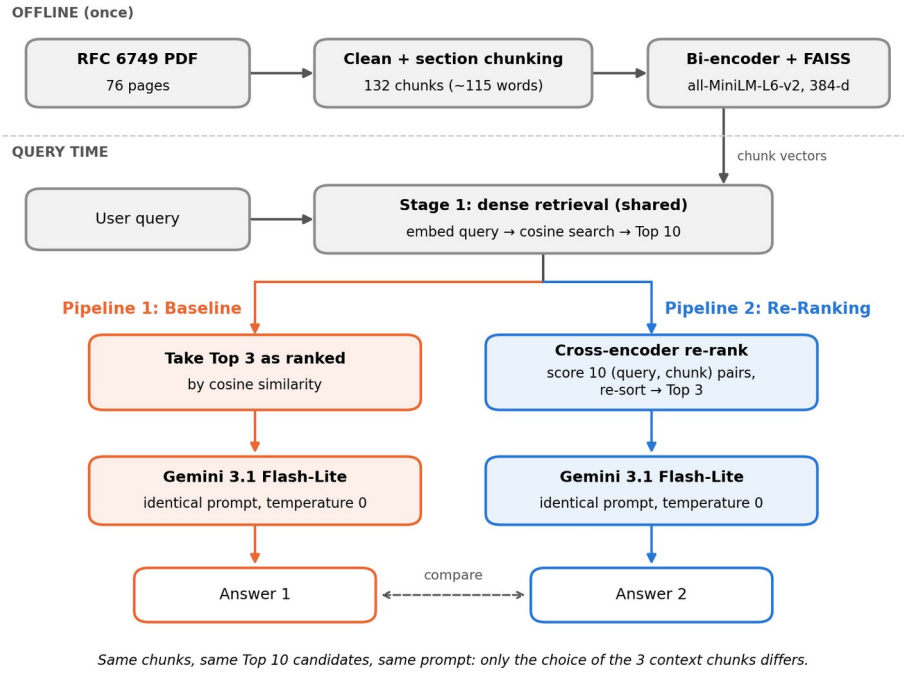


Figure 1. The two pipelines share the chunks, the FAISS index and the Top 10 candidates; Pipeline 2 adds a cross-encoder re-ranking step before choosing the 3 chunks sent to Gemini.

Bi-encoder (Pipeline 1). The query and each chunk are encoded *separately* into one 384-dimensional vector each, and relevance is the cosine similarity between them. Because a chunk's vector does not depend on the question, I embedded all 132 chunks once and stored them in FAISS, so a query only needs one encoding and a vector search (~12 ms). The downside is that the query and chunk never interact: the vector mostly captures what a chunk is *about*, and the words that separate RFC rules ("MUST NOT", "SHOULD NOT", which grant type) barely move it.

Cross-encoder (Pipeline 2). The query and chunk are fed *together* ([CLS] query [SEP] chunk [SEP]), so self-attention lets every query token look at every chunk token in every layer, and the model outputs one relevance score for the pair. It can check whether a chunk actually answers the question (e.g. it linked "a client acting on its own behalf" to the client credentials grant in Section 4.4.3). The

cost is that nothing can be precomputed: every (query, chunk) pair needs a full forward pass at query time.

	Bi-Encoder	Cross-Encoder
Input / interaction	Encoded separately; one cosine score at the end	Encoded together; attention across all tokens
Precompute?	Yes, chunk vectors stored in a vector index	No, one model pass per pair per query
Speed (my run)	~12 ms for the whole corpus	~46 ms per pair
Strength / role	High recall, scalable: Stage 1 candidates	High precision: Stage 2 re-ranking

2. Qualitative & Rank-Shift Analysis

2.1 Rank shifts of the correct chunk

#	Failure type	Query	Baseline rank	Re-ranked rank
Q1	Keyword Trap / Distractor	Must access tokens be kept confidential in transit and storage?	#4	**#1**
Q2	Exceptions & Corner Cases	In the client credentials flow, should the access token response contain a refresh token?	#8	**#1**
Q3	Keyword Trap (synonym / role)	Does the token response for a client acting on its own behalf include a refresh token?	#9	**#1**
Q4	Negation & Constraints	Can a refreshed access token have a broader scope than the original one?	#3	**#2**
Q5	Other: Buried chunk	Which parameters are required in the authorization request?	#5	**#3**
Q6	Limitation: re-ranking demotes the answer	Must the token endpoint response include Cache-Control no-store?	#1	**#4**
Q7	Control: rule stated in two places	In the implicit flow, can the access token response include a refresh token?	#3	**#2**

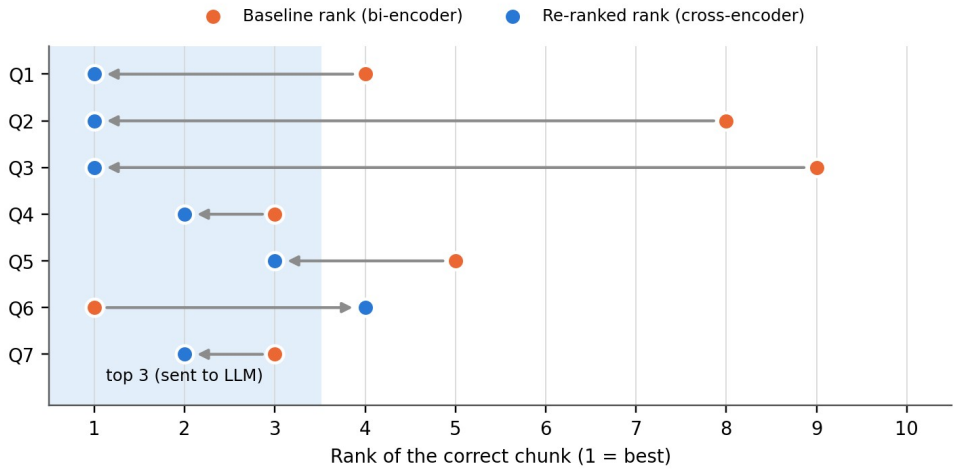


Figure 2. Rank of the correct chunk before (orange) and after (blue) re-ranking. Only the shaded top 3 are sent to the LLM.

2.2 Answer 1 (baseline) vs Answer 2 (re-ranked)

Answer 1 was correct for **3 of 7** queries, Answer 2 for **6 of 7**. Full answers are saved in the notebook and in results/stress_test_summary.csv.

#	Answer 1	Answer 2	What happened
Q1	Incomplete	**Correct**	Baseline only had s.10.8 (no sending in the clear); it missed 'storage' and who may see the token. Answer 2 had s.10.3 at #1 and gave the full rule.
Q2	Omitted	**Correct**	Baseline only saw the general refresh-token text (s.1.5) and could not answer; Answer 2 quotes s.4.4.3: 'A refresh token SHOULD NOT be included'.
Q3	Omitted	**Correct**	Same rule asked with different words. Baseline could not connect 'acting on its own behalf' with the client credentials grant; Answer 2 did (s.4.4.3).
Q4	Correct	**Correct**	s.6 was already #3 for the baseline, so both answers cite the MUST NOT rule; re-ranking only moved it up to #2.
Q5	Misstated	**Correct**	Baseline gave the implicit-grant parameters (response_type=token) as the general answer; Answer 2 explains that it depends on the grant and gives response_type=code for s.4.1.1.
Q6	Correct	**Hedged**	Re-ranking pushed the rule (s.5.1) from #1 to #4 in favour of example responses, so Answer 2 became unsure while Answer 1 was correct.
Q7	Correct	**Correct**	The rule is written in s.4.2 and s.4.2.2; s.4.2 was in the top 3 for both pipelines, so both answers were correct.

Example answers. The three cases below show the three different outcomes word for word.

Q2. In the client credentials flow, should the access token response contain a refresh token?
Omitted rule | correct chunk #8 → #1

Answer 1 (baseline): Omitted	**Answer 2 (re-ranked): Correct**
The provided context does not answer this. While it notes that issuing a refresh token is optional at the discretion of the authorization server [1], it does not specify the requirements or behavior regarding refresh tokens within the client credentials flow.	In the client credentials flow, a refresh token should not be included in the access token response. According to Section 4.4.3, if the access token request is valid and authorized, the authorization server issues an access token but a refresh token SHOULD NOT be included.

Q5. Which parameters are required in the authorization request?
Misstated rule | correct chunk #5 → #3

Answer 1 (baseline): Misstated	**Answer 2 (re-ranked): Correct**
The required parameters for the authorization request are `response_type` (which must be set to "token") and `client_id` (the client identifier). These are specified in Section 4.2.1.	The required parameters for an authorization request depend on the grant type. For the authorization code grant, the required parameters are `response_type` (set to "code") and `client_id` (Section 4.1.1). For the implicit grant, the required parameters are `response_type` (set to "token") and `client_id` (Section 4.2.1).

Q6. Must the token endpoint response include Cache-Control no-store? Re-ranking made it worse | correct chunk #1 → #4	
Answer 1 (baseline): Correct Yes, the authorization server must include the HTTP "Cache-Control" response header field with a value of "no-store" in any response containing tokens, credentials, or other sensitive information [1].	**Answer 2 (re-ranked): Hedged** The provided context does not explicitly state that the response *must* include "Cache-Control: no-store". However, the example successful responses provided in sections 4.1.4, 4.3.3, and 4.4.3 all include "Cache-Control: no-store" as part of the HTTP response headers.

2.3 Discussion

The bi-encoder matched the topic, not the rule. In Q2 and Q3 the phrase "refresh token" pulled the general Section 1.5 *Refresh Token* chunks to ranks #1–#3, while the actual rule (Section 4.4.3: "A refresh token SHOULD NOT be included" for the client credentials grant) sat at #8 and #9. In Q1 "access tokens" matched the definition in Section 1.4, while the confidentiality rule is in the Security Considerations (Section 10.3, #4). Negation on its own was not the problem: direct "MUST NOT" questions in the spec's wording (Q4 and others I tried) were usually found. Synonyms were the hardest case: describing the grant as "a client acting on its own behalf" dropped Section 4.4.3 to #9, but the cross-encoder still put it at #1.

The baseline omitted, half-stated and once misstated rules. In Q2 and Q3 it said "the provided context does not answer this"; in Q1 it gave only the "not in the clear" half of the rule. In Q5 it was actually wrong: its context only had the implicit-grant request (Section 4.2.1), so it said the required parameters are response_type = "token" and client_id, as if that were the general rule. Answer 2 had Section 4.1.1 and correctly explained that it depends on the grant. Gemini never invented facts (the prompt restricts it to the context), so all differences come from retrieval.

Re-ranking can also hurt. In Q6 the bi-encoder had the Cache-Control rule (Section 5.1) at #1, but the cross-encoder preferred the example responses that literally contain Cache-Control: no-store and pushed the rule to #4, so Answer 2 hedged. The re-ranker is trained on web search data (MS MARCO) and can be fooled by text that only looks like the answer. Sending more chunks to the LLM, hybrid BM25 + dense retrieval, or a domain-tuned re-ranker could reduce such errors.

3. Latency Profiling

7 queries × 20 repetitions, 132 chunks, CPU only, models pre-loaded.

Stage	Mean (ms)	Median (ms)	p95 (ms)
Pipeline 1: bi-encoder top-10 (retrieval only)	12.4	12.2	15.3
Cross-encoder re-rank of 10 pairs	628.8	670.0	774.2
Pipeline 2: bi-encoder + re-rank (retrieval only)	641.1	680.9	787.4
Gemini answer generation (per call)	1,550.3	1,377.2	2,259.3

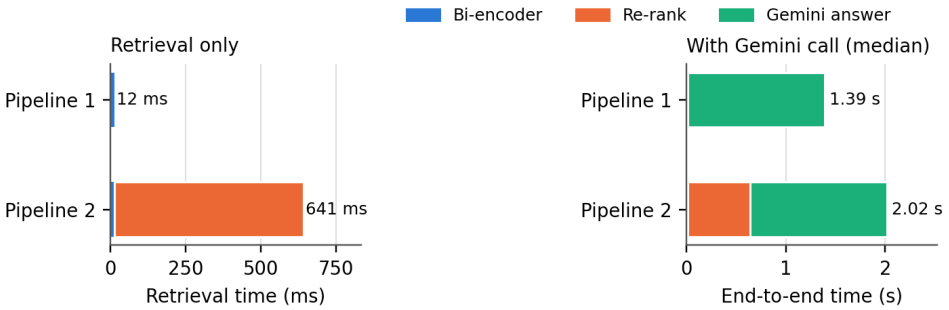


Figure 3. Mean retrieval time (left) and end-to-end time with the median Gemini call (right).

Re-ranking 10 pairs took ~629 ms versus ~12 ms for the whole bi-encoder retrieval, so Pipeline 2's retrieval is about **52× slower**, because each pair is a full transformer pass over ~150–200 tokens. With Gemini Flash-Lite answering in ~1.4 s, re-ranking adds about **45%** to the total time (≈1.4 s → ≈2.0 s). For that extra ~0.6 s, correct answers went from 3/7 to 6/7, which I think is worth it for a specification where one "SHOULD NOT" changes the answer; on a GPU the gap would be much smaller.

4. Scaling Question

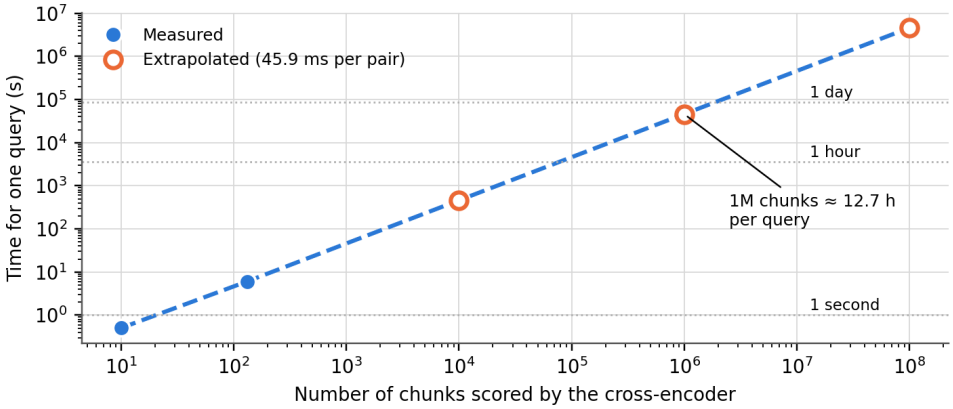


Figure 4. Cross-encoder time for one query vs. number of chunks (log-log). Blue = measured, orange = extrapolated.

A cross-encoder must read every (query, chunk) pair together, so nothing can be prepared in advance: each new question needs one full model pass per chunk, and the cost grows linearly with the corpus. In my run one pair took ~46 ms; scoring all 132 chunks already took ~6.1 s per query (vs ~11 ms for the bi-encoder). At that speed 1 million chunks would take **~12.7 hours per question** and 100 million about 53 days. Bi-encoder vectors, in contrast, are computed once offline and searched with a fast index (FAISS, or HNSW/IVF at scale). That is why RAG uses two stages: the cheap bi-encoder narrows the collection to a few candidates with high recall, and the expensive cross-encoder only spends its compute on those few, giving most of its accuracy at a small, fixed extra cost.